

SHOPSY E-COMMERCE DATABASE MANAGEMENT SYSTEM

SELLER AND INVENTORY MANAGEMENT SYSTEM

1.Database Design :

The ShopSphere Online E-Commerce Database Management System uses Seller, Product, and Inventory as important entities for managing sellers, products, and stock available on the e-commerce platform. The Seller table stores information about different sellers, while the Product table stores product details such as product name, seller ID, category ID, price, stock, and description. The Inventory table stores available stock and last updated information for each product. The Product table is connected to the Seller table using Seller_ID, and the Inventory table is connected to the Product table using Product_ID, which establishes one-to-many relationships between the related tables.

1.1 Seller Table :

Attribute	Key Type
Seller_ID	Primary Key
Seller_Name	Not Null
Email	Unique Key
Phone	-
Address	-

1.2 Inventory Table :

Attribute	Key Type
Inventory_ID	Primary Key
Product_ID	Foreign Key
Available_Stock	-
Last_Updated	-

2. Relationship :

Seller → Inventory

- Seller.Seller_ID → Primary Key
- Inventory.Inventory_ID → Primary Key
- Inventory.Product_ID → Foreign Key referencing Product.Product_ID
- Relationship: One-to-Many (1:M)
- One seller can have multiple inventory records.

3. Table Creation :

3.1 Create Seller Table :

```
SQL> CREATE TABLE Seller (  
2     Seller_ID NUMBER PRIMARY KEY,  
3     Seller_Name VARCHAR2(50) NOT NULL,  
4     Email VARCHAR2(50) UNIQUE,  
5     Phone VARCHAR2(15),  
6     Address VARCHAR2(100)  
7 );
```

Table created.

3.2 Create Inventory Table :

```
SQL> CREATE TABLE Inventory (  
2     Inventory_ID NUMBER PRIMARY KEY,  
3     Product_ID NUMBER,  
4     Available_Stock NUMBER,  
5     Last_Updated DATE,  
6     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
7 );
```

Table created.

4.Data Insertion :

4.1 Seller Records :

```
SQL> INSERT INTO Seller VALUES (1, 'ABC Traders', 'abc@gmail.com', '9876543210', 'Chennai');  
1 row created.  
  
SQL> INSERT INTO Seller VALUES (2, 'Siva Stores', 'siva@gmail.com', '9876543211', 'Chromepet');  
1 row created.  
  
SQL> INSERT INTO Seller VALUES (3, 'Sri Enterprises', 'sri@gmail.com', '9876543212', 'Tambaram');  
1 row created.  
  
SQL> INSERT INTO Seller VALUES (4, 'Kumar Mart', 'kumar@gmail.com', '9876543213', 'Pallavaram');  
1 row created.  
  
SQL> INSERT INTO Seller VALUES (5, 'Ravi Products', 'ravi@gmail.com', '9876543214', 'Guindy');  
1 row created.  
  
SQL> INSERT INTO Seller VALUES (6, 'Green Shop', 'green@gmail.com', '9876543215', 'Velachery');  
1 row created.
```

4.2 Inventory Records :

```
SQL> INSERT INTO Inventory VALUES (1001, 101, 20, SYSDATE);
1 row created.

SQL> INSERT INTO Inventory VALUES (1002, 102, 30, SYSDATE);
1 row created.

SQL> INSERT INTO Inventory VALUES (1003, 103, 50, SYSDATE);
1 row created.

SQL> INSERT INTO Inventory VALUES (1004, 104, 40, SYSDATE);
1 row created.

SQL> INSERT INTO Inventory VALUES (1005, 105, 60, SYSDATE);
1 row created.

SQL> INSERT INTO Inventory VALUES (1006, 106, 15, SYSDATE);
1 row created.
```

5. Data Retrieval :

5.1 Display Seller Records :

```
SQL> SELECT * FROM Seller;

SELLER_ID SELLER_NAME
-----
EMAIL                                           PHONE
-----
ADDRESS
-----
          1 ABC Traders
abc@gmail.com                                9876543210
Chennai

          2 Siva Stores
siva@gmail.com                              9876543211
Chromepet

SELLER_ID SELLER_NAME
-----
EMAIL                                           PHONE
-----
ADDRESS
-----
          3 Sri Enterprises
sri@gmail.com                                9876543212
Tambaram
```

```
4 Kumar Mart
kumar@gmail.com 9876543213
```

```
SELLER_ID SELLER_NAME
```

```
EMAIL PHONE
```

```
ADDRESS
```

```
Pallavaram
```

```
5 Ravi Products
ravi@gmail.com 9876543214
Guindy
```

```
6 Green Shop
```

```
SELLER_ID SELLER_NAME
```

```
EMAIL PHONE
```

```
ADDRESS
```

```
green@gmail.com 9876543215
Velachery
```

```
6 rows selected.
```

5.2 Display Inventory Records :

```
SQL> SELECT * FROM Inventory;
```

```
INVENTORY_ID PRODUCT_ID AVAILABLE_STOCK LAST_UPDA
```

```
-----
1001          101          20 18-SEP-26
1002          102          30 18-SEP-26
1003          103          50 18-SEP-26
1004          104          40 18-SEP-26
1005          105          60 18-SEP-26
1006          106          15 18-SEP-26
```

```
6 rows selected.
```

6.Seller Product Information :

```
SQL> SELECT
2      Seller.Seller_Name,
3      Product.Product_Name,
4      Product.Product_ID,
5      Product.Price,
6      Product.Stock
7  FROM Seller
8  JOIN Product
9  ON Seller.Seller_ID = Product.Seller_ID;
```

SELLER_NAME			
PRODUCT_NAME	PRODUCT_ID	PRICE	
STOCK			
ABC Traders Laptop 20	101	55000	
Siva Stores Smartphone 30	102	25000	

SELLER_NAME			
PRODUCT_NAME	PRODUCT_ID	PRICE	
STOCK			
Sri Enterprises Headphones 50	103	2500	

```

Kumar Mart
Keyboard                                104      1500
SELLER_NAME
-----
PRODUCT_NAME                          PRODUCT_ID  PRICE
-----
      STOCK
-----
      40

Ravi Products
Mouse                                105      800
      60

Green Shop
SELLER_NAME
-----
PRODUCT_NAME                          PRODUCT_ID  PRICE
-----
      STOCK
-----
Monitor                                106     12000
      15

6 rows selected.

```

7. Product Availability :

7.1 Available Products :

```

SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      i.Available_Stock
  5 FROM Product p
  6 JOIN Inventory i
  7 ON p.Product_ID = i.Product_ID
  8 WHERE i.Available_Stock > 0;

```

PRODUCT_ID	PRODUCT_NAME	AVAILABLE_STOCK
101	Laptop	20
102	Smartphone	30
103	Headphones	50
104	Keyboard	40
105	Mouse	60
106	Monitor	15

6 rows selected.

7.2 Unavailable Products :

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.Available_Stock,
  5     'Out of Stock' AS Stock_Status
  6 FROM Product p
  7 JOIN Inventory i
  8 ON p.Product_ID = i.Product_ID
  9 WHERE i.Available_Stock = 0;

no rows selected
```

8. Inventory Update :

```
SQL> UPDATE Inventory
  2 SET Available_Stock = 35
  3 WHERE Inventory_ID = 1001;

1 row updated.
```

```
SQL> SELECT * FROM Inventory;
```

INVENTORY_ID	PRODUCT_ID	AVAILABLE_STOCK	LAST_UPDA
1001	101	35	18-SEP-26
1002	102	30	18-SEP-26
1003	103	50	18-SEP-26
1004	104	40	18-SEP-26
1005	105	60	18-SEP-26
1006	106	15	18-SEP-26

```
6 rows selected.
```

9. Inventory Status Reports :

9.1 Complete Inventory Report :

```
SQL> SELECT
  2     i.Inventory_ID,
  3     p.Product_Name,
  4     s.Seller_Name,
  5     i.Available_Stock,
  6     CASE
  7         WHEN i.Available_Stock > 0 THEN 'Available'
  8         ELSE 'Out of Stock'
  9     END AS Stock_Status,
 10     i.Last_Updated
 11 FROM Inventory i
 12 JOIN Product p
 13 ON i.Product_ID = p.Product_ID
 14 JOIN Seller s
 15 ON p.Seller_ID = s.Seller_ID;
```

```

INVENTORY_ID PRODUCT_NAME
-----
SELLER_NAME AVAILABLE_STOCK STOCK_STATUS
-----
LAST_UPDA
-----
          1001 Laptop
ABC Traders 35 Available
18-SEP-26

          1002 Smartphone
Siva Stores 30 Available
18-SEP-26

INVENTORY_ID PRODUCT_NAME
-----
SELLER_NAME AVAILABLE_STOCK STOCK_STATUS
-----
LAST_UPDA
-----
          1003 Headphones
Sri Enterprises 50 Available
18-SEP-26

          1004 Keyboard
Kumar Mart 40 Available

```

```

INVENTORY_ID PRODUCT_NAME
-----
SELLER_NAME AVAILABLE_STOCK STOCK_STATUS
-----
LAST_UPDA
-----
18-SEP-26

          1005 Mouse
Ravi Products 60 Available
18-SEP-26

          1006 Monitor

INVENTORY_ID PRODUCT_NAME
-----
SELLER_NAME AVAILABLE_STOCK STOCK_STATUS
-----
LAST_UPDA
-----
Green Shop 15 Available
18-SEP-26

6 rows selected.

```

9.2 Total Stock by Seller :

```
SQL> SELECT
  2     s.Seller_Name,
  3     SUM(i.Available_Stock) AS Total_Stock
  4 FROM Seller s
  5 JOIN Product p
  6 ON s.Seller_ID = p.Seller_ID
  7 JOIN Inventory i
  8 ON p.Product_ID = i.Product_ID
  9 GROUP BY s.Seller_Name;
```

SELLER_NAME	TOTAL_STOCK
ABC Traders	35
Siva Stores	30
Sri Enterprises	50
Kumar Mart	40
Ravi Products	60
Green Shop	15

6 rows selected.

9.3 Available and Unavailable Product Count :

```
SQL> SELECT
  2     CASE
  3         WHEN Available_Stock > 0 THEN 'Available'
  4         ELSE 'Out of Stock'
  5     END AS Stock_Status,
  6     COUNT(*) AS Product_Count
  7 FROM Inventory
  8 GROUP BY
  9     CASE
 10         WHEN Available_Stock > 0 THEN 'Available'
 11         ELSE 'Out of Stock'
 12     END;
```

STOCK_STATUS	PRODUCT_COUNT
Available	6

Conclusion :

The ShopSphere Online E-Commerce Database Management System successfully manages seller, product, and inventory information using SQL. The system uses primary keys and foreign keys to establish relationships between the related tables. SQL operations such as CREATE, INSERT, SELECT, JOIN, UPDATE, and DELETE are used to manage the data. The system also helps to maintain available stock information, identify unavailable products, and generate inventory status reports. Overall, the database provides an organized way to manage seller, product, and inventory details in the e-commerce platform.